```
import { useState, useEffect, useRef } from "react";

// ── Palette ──────────────────────────────────────────────────────────
const C = {
  void: "#050508",
  deep: "#0a0b0f",
  titanium: "#12141a",
  chrome: "#8a9aaa",
  chromeDim: "#2a3040",
  pearl: "#d4e8f0",
  pearlDim: "#7ba8bc",
  accent: "#4dd9ff",
  gold: "#c8a84b",
  goldDim: "#3a2e10",
  goldBright: "#f0d070",
  red: "#ff4455",
  green: "#44ffaa",
  white: "#f0f4f8",
  artifact: "#c8a84b",
  canon: "#4dd9ff",
};

// ── QR Code Generator (pure JS, no library) ──────────────────────────
function generateQRMatrix(data) {
  // Simplified QR-like matrix for visual representation
  // In production, use a real QR library
  const size = 21;
  const matrix = Array(size).fill(null).map(() => Array(size).fill(0));

  // Position detection patterns
  const addFinder = (row, col) => {
    for (let r = 0; r < 7; r++) {
      for (let c = 0; c < 7; c++) {
        if (r === 0 || r === 6 || c === 0 || c === 6 || (r >= 2 && r <= 4 && c >=
          if (row + r < size && col + c < size) matrix[row + r][col + c] = 1;
        }
      }
    }
  };
  addFinder(0, 0);
  addFinder(0, 14);
  addFinder(14, 0);

  // Timing patterns
```

```
  for (let i = 8; i < 13; i++) {
    matrix[6][i] = i % 2 === 0 ? 1 : 0;
    matrix[i][6] = i % 2 === 0 ? 1 : 0;
  }

  // Seed data into remaining cells
  let hash = 0;
  for (let i = 0; i < data.length; i++) {
    hash = ((hash << 5) - hash) + data.charCodeAt(i);
    hash |= 0;
  }

  const rng = (seed) => {
    let s = seed;
    return () => {
      s = (s * 1664525 + 1013904223) & 0xffffffff;
      return (s >>> 0) / 0xffffffff;
    };
  };
  const rand = rng(Math.abs(hash));

  for (let r = 0; r < size; r++) {
    for (let c = 0; c < size; c++) {
      if (matrix[r][c] === 0 && !(r < 8 && c < 8) && !(r < 8 && c > 12) && !(r >
        matrix[r][c] = rand() > 0.5 ? 1 : 0;
      }
    }
  }

  return matrix;
}

function applyArtifactFracture(matrix, sessionId) {
  // Deliberate single imperfection seeded from sessionId
  let seed = 0;
  for (let i = 0; i < sessionId.length; i++) {
    seed = ((seed << 5) - seed) + sessionId.charCodeAt(i);
    seed |= 0;
  }
  const x = 8 + (Math.abs(seed) % 6);
  const y = 8 + (Math.abs(seed * 31) % 6);
  const fractured = matrix.map(row => [...row]);
  if (fractured[y] && fractured[y][x] !== undefined) {
    fractured[y][x] = fractured[y][x] === 1 ? 2 : 1; // 2 = fracture mark
  }
  return fractured;
}
```

```
// ── Fingerprint Generator ──────────────────────────────────────────
function generateFingerprint(str) {
  let h1 = 0xdeadbeef, h2 = 0x41c6ce57;
  for (let i = 0; i < str.length; i++) {
    const ch = str.charCodeAt(i);
    h1 = Math.imul(h1 ^ ch, 2654435761);
    h2 = Math.imul(h2 ^ ch, 1597334677);
  }
  h1 = Math.imul(h1 ^ (h1 >>> 16), 2246822507) ^ Math.imul(h2 ^ (h2 >>> 13), 3266
  h2 = Math.imul(h2 ^ (h2 >>> 16), 2246822507) ^ Math.imul(h1 ^ (h1 >>> 13), 3266
  const hex = (4294967296 * (2097151 & h2) + (h1 >>> 0)).toString(16).toUpperCase
  return hex.padStart(12, '0').match(/.{1,4}/g).join('-');
}

// ── QR Display Component ──────────────────────────────────────────
function QRDisplay({ data, isArtifact, sessionId, size = 80 }) {
  const matrix = generateQRMatrix(data);
  const display = isArtifact ? applyArtifactFracture(matrix, sessionId) : matrix;
  const cellSize = size / display.length;

  return (
    <svg width={size} height={size} style={{ display: 'block' }}>
      <rect width={size} height={size} fill="white" />
      {display.map((row, r) =>
        row.map((cell, c) => {
          if (cell === 0) return null;
          const fill = cell === 2 ? '#8a6a1a' : '#050508'; // fracture = gold-bro
          return (
            <rect
              key={`${r}-${c}`}
              x={c * cellSize}
              y={r * cellSize}
              width={cellSize}
              height={cellSize}
              fill={fill}
            />
          );
        })
      )}
      {isArtifact && (
        <rect
          x={0} y={size - 2}
          width={size} height={2}
          fill={C.gold}
          opacity={0.6}
        />
```

```
      )}
    </svg>
  );
}

// ── Card Record Component ──────────────────────────────────────────
function CardRecord({ record, onSelect, isSelected }) {
  const isArtifact = record.class === 'Artifact';
  const classColor = isArtifact ? C.gold : C.accent;

  return (
    <div
      onClick={() => onSelect(record)}
      style={{
        background: isSelected ? (isArtifact ? 'rgba(200,168,75,0.08)' : 'rgba(77
        border: `1px solid ${isSelected ? classColor : C.chromeDim}`,
        borderRadius: 4,
        padding: '12px 14px',
        cursor: 'pointer',
        transition: 'all 0.2s',
        display: 'flex',
        alignItems: 'center',
        gap: 12,
        marginBottom: 6,
      }}
    >
      <div style={{
        fontSize: 7,
        fontFamily: "'Courier New', monospace",
        color: classColor,
        letterSpacing: 2,
        fontStyle: 'italic',
        minWidth: 48,
      }}>
        {record.class}
      </div>
      <div style={{ flex: 1 }}>
        <div style={{
          fontSize: 11,
          fontFamily: "'Courier New', monospace",
          color: C.pearl,
          letterSpacing: 1,
          marginBottom: 2,
        }}>
          {record.character}
        </div>
        <div style={{
```

```
          fontSize: 8,
          fontFamily: "'Courier New', monospace",
          color: C.chrome,
          letterSpacing: 1,
        }}>
          {record.sessionId} · v{record.canonVersion}
        </div>
      </div>
      <div style={{
        fontSize: 7,
        fontFamily: "'Courier New', monospace",
        color: C.chromeDim,
        letterSpacing: 1,
        textAlign: 'right',
      }}>
        {new Date(record.date).toLocaleDateString('en-US', { month: 'short', day:
      </div>
      <div style={{
        width: 6, height: 6,
        borderRadius: '50%',
        background: record.status === 'released' ? C.green : record.status === 'p
      }} />
    </div>
  );
}

// ── Delta Log Entry ──────────────────────────────────────────────────
function DeltaEntry({ delta }) {
  return (
    <div style={{
      borderLeft: `2px solid ${C.goldDim}`,
      paddingLeft: 10,
      marginBottom: 8,
      paddingTop: 2,
      paddingBottom: 2,
    }}>
      <div style={{
        fontSize: 8,
        fontFamily: "'Courier New', monospace",
        color: C.gold,
        letterSpacing: 1,
        marginBottom: 2,
      }}>
        Canon v{delta.canonVersion} · {new Date(delta.date).toLocaleDateString()}
      </div>
      <div style={{
        fontSize: 9,
```

```
        fontFamily: "'Courier New', monospace",
        color: C.chrome,
        letterSpacing: 0.5,
        lineHeight: 1.5,
      }}>
        {delta.note}
      </div>
    </div>
  );
}

// ── Main Registry ──────────────────────────────────────────────────────
const INITIAL_RECORDS = [
  {
    id: 'FGV-001',
    character: 'ISOLDE',
    sessionId: 'SB001-ISO-2026-001',
    class: 'Canon',
    canonVersion: '1.0',
    date: '2026-05-18T00:00:00',
    status: 'prerelease',
    laneStatus: { l1: true, l2: true, l3: false, l4: false },
    notes: 'Initial seed drop. Lead character for Gumroad launch.',
    deltaLog: [],
    fingerprint: null,
  },
  {
    id: 'FGV-002',
    character: 'RAVEN SB_330',
    sessionId: 'SB330-RAV-2025-118',
    class: 'Artifact',
    canonVersion: '1.2',
    date: '2025-05-18T00:00:00',
    status: 'released',
    laneStatus: { l1: true, l2: true, l3: true, l4: false },
    notes: 'Competing lanes detected post-session. Unreproducible configuration.
    deltaLog: [
      { canonVersion: '1.1', date: '2025-06-01T00:00:00', note: 'Lane 2/Lane 1 se
      { canonVersion: '1.2', date: '2025-08-15T00:00:00', note: 'Gain threshold m
    ],
    fingerprint: null,
  },
  {
    id: 'FGV-003',
    character: 'CALISTA VOSS',
    sessionId: 'SB200-CAL-2026-044',
    class: 'Canon',
```

```
    canonVersion: '1.0',
    date: '2026-03-22T00:00:00',
    status: 'released',
    laneStatus: { l1: true, l2: true, l3: true, l4: false },
    notes: 'Clean lane separation. Canon lock achieved at 0.96.',
    deltaLog: [],
    fingerprint: null,
  },
];

// Inject fingerprints
INITIAL_RECORDS.forEach(r => {
  r.fingerprint = generateFingerprint(`${r.character}${r.sessionId}${r.date}`);
});

export default function FGEVaultRegistry() {
  const [records, setRecords] = useState(INITIAL_RECORDS);
  const [selected, setSelected] = useState(INITIAL_RECORDS[0]);
  const [tab, setTab] = useState('registry'); // registry | new | inspect
  const [filter, setFilter] = useState('all'); // all | canon | artifact
  const [boot, setBoot] = useState(false);
  const [tick, setTick] = useState(0);

  // New record form state
  const [form, setForm] = useState({
    character: '',
    sessionId: '',
    class: 'Canon',
    canonVersion: '1.0',
    notes: '',
    laneStatus: { l1: false, l2: false, l3: false, l4: false },
  });

  useEffect(() => {
    setTimeout(() => setBoot(true), 300);
    const interval = setInterval(() => setTick(t => t + 1), 1000);
    return () => clearInterval(interval);
  }, []);

  const filteredRecords = records.filter(r =>
    filter === 'all' ? true : r.class.toLowerCase() === filter
  );

  const handleSubmit = () => {
    if (!form.character || !form.sessionId) return;
    const now = new Date().toISOString();
    const newRecord = {
```

```
    id: `FGV-${String(records.length + 1).padStart(3, '0')}`,
    character: form.character.toUpperCase(),
    sessionId: form.sessionId,
    class: form.class,
    canonVersion: form.canonVersion,
    date: now,
    status: 'prerelease',
    laneStatus: form.laneStatus,
    notes: form.notes,
    deltaLog: [],
    fingerprint: generateFingerprint(`${form.character}${form.sessionId}${now}`
  };
  setRecords(prev => [newRecord, ...prev]);
  setSelected(newRecord);
  setTab('registry');
  setForm({
    character: '',
    sessionId: '',
    class: 'Canon',
    canonVersion: '1.0',
    notes: '',
    laneStatus: { l1: false, l2: false, l3: false, l4: false },
  });
};

const handleStatusChange = (status) => {
  setRecords(prev => prev.map(r =>
    r.id === selected.id ? { ...r, status } : r
  ));
  setSelected(prev => ({ ...prev, status }));
};

const laneLabels = {
  l1: 'L1 · Identity Anchor',
  l2: 'L2 · Gain/State',
  l3: 'L3 · Scene/Environment',
  l4: 'L4 · Escalation Vector',
};

const isArtifact = selected?.class === 'Artifact';
const classColor = isArtifact ? C.gold : C.accent;

return (
  <div style={{
    background: C.void,
    minHeight: '100vh',
    fontFamily: "'Courier New', monospace",
```

```
  color: C.pearl,
  display: 'flex',
  flexDirection: 'column',
  opacity: boot ? 1 : 0,
  transition: 'opacity 0.8s',
}}>

  {/* Classification bands */}
  <div style={{ height: 4, background: `repeating-linear-gradient(90deg, ${C.

  {/* Header */}
  <div style={{
    borderBottom: `1px solid ${C.chromeDim}`,
    padding: '16px 24px',
    display: 'flex',
    alignItems: 'center',
    justifyContent: 'space-between',
    background: C.deep,
  }}>
    <div>
      <div style={{ fontSize: 9, color: C.gold, letterSpacing: 4, marginBotto
        FERAL GLOSS EMPIRE
      </div>
      <div style={{ fontSize: 18, color: C.white, letterSpacing: 3, fontWeigh
        VAULT REGISTRY
      </div>
      <div style={{ fontSize: 7, color: C.chromeDim, letterSpacing: 3, margin
        AUTHENTICATION · PROVENANCE · CANON STANDARD v2.1
      </div>
    </div>
    <div style={{ textAlign: 'right' }}>
      <div style={{ fontSize: 7, color: C.chromeDim, letterSpacing: 2, margin
        CANON ENGINE v2.1
      </div>
      <div style={{
        fontSize: 11,
        color: C.green,
        letterSpacing: 2,
        fontFamily: "'Courier New', monospace",
      }}>
        ● SYSTEM ACTIVE
      </div>
      <div style={{ fontSize: 7, color: C.chromeDim, letterSpacing: 2, margin
        CYCLE {String(tick).padStart(6, '0')}
      </div>
    </div>
  </div>
```

```
{/* Tab Bar */}
<div style={{
  display: 'flex',
  borderBottom: `1px solid ${C.chromeDim}`,
  background: C.deep,
}}>
  {[
    { key: 'registry', label: 'REGISTRY' },
    { key: 'new', label: '+ NEW RECORD' },
    { key: 'inspect', label: 'INSPECT' },
  ].map(t => (
    <button
      key={t.key}
      onClick={() => setTab(t.key)}
      style={{
        background: 'none',
        border: 'none',
        borderBottom: tab === t.key ? `2px solid ${C.accent}` : '2px solid
        color: tab === t.key ? C.white : C.chromeDim,
        fontSize: 8,
        letterSpacing: 3,
        padding: '12px 20px',
        cursor: 'pointer',
        fontFamily: "'Courier New', monospace",
        transition: 'all 0.2s',
      }}
    >
      {t.label}
    </button>
  ))}
  <div style={{ flex: 1 }} />
  <div style={{
    display: 'flex',
    alignItems: 'center',
    gap: 8,
    padding: '0 20px',
  }}>
    {['all', 'canon', 'artifact'].map(f => (
      <button
        key={f}
        onClick={() => setFilter(f)}
        style={{
          background: filter === f ? (f === 'artifact' ? C.goldDim : f ===
          border: `1px solid ${f === 'artifact' ? C.goldDim : f === 'canon'
          color: filter === f ? (f === 'artifact' ? C.gold : f === 'canon'
          fontSize: 7,
```

```
            letterSpacing: 2,
            padding: '4px 10px',
            cursor: 'pointer',
            fontFamily: "'Courier New', monospace",
            borderRadius: 2,
            fontStyle: f !== 'all' ? 'italic' : 'normal',
          }}
        >
          {f.toUpperCase()}
        </button>
      ))}
    </div>
  </div>

  {/* Main Content */}
  <div style={{ flex: 1, display: 'flex', overflow: 'hidden' }}>

    {/* Left Panel — Record List */}
    {tab !== 'new' && (
      <div style={{
        width: 300,
        borderRight: `1px solid ${C.chromeDim}`,
        overflow: 'auto',
        padding: 16,
        background: C.deep,
      }}>
        <div style={{
          fontSize: 7,
          color: C.chromeDim,
          letterSpacing: 3,
          marginBottom: 12,
        }}>
          {filteredRecords.length} RECORDS · {records.filter(r => r.class ===
        </div>
        {filteredRecords.map(r => (
          <CardRecord
            key={r.id}
            record={r}
            onSelect={setSelected}
            isSelected={selected?.id === r.id}
          />
        ))}
      </div>
    )}

    {/* Right Panel — Detail / New Form */}
    <div style={{ flex: 1, overflow: 'auto', padding: 24 }}>
```

```
{/* NEW RECORD FORM */}
{tab === 'new' && (
  <div style={{ maxWidth: 600 }}>
    <div style={{ fontSize: 9, color: C.gold, letterSpacing: 4, marginB
      NEW CARD RECORD · QUALITY GATE
    </div>

    {/* Class Selection */}
    <div style={{ marginBottom: 20 }}>
      <div style={{ fontSize: 7, color: C.chromeDim, letterSpacing: 2,
        CLASSIFICATION
      </div>
      <div style={{ display: 'flex', gap: 10 }}>
        {['Canon', 'Artifact'].map(cls => (
          <button
            key={cls}
            onClick={() => setForm(f => ({ ...f, class: cls }))}
            style={{
              background: form.class === cls
                ? (cls === 'Artifact' ? 'rgba(200,168,75,0.15)' : 'rgba
                : C.titanium,
              border: `1px solid ${form.class === cls ? (cls === 'Artif
              color: form.class === cls ? (cls === 'Artifact' ? C.gold
              fontSize: 10,
              letterSpacing: 3,
              padding: '10px 24px',
              cursor: 'pointer',
              fontFamily: "'Courier New', monospace",
              fontStyle: 'italic',
              borderRadius: 3,
              transition: 'all 0.2s',
            }}
          >
            {cls}
          </button>
        ))}
      </div>
      {form.class === 'Artifact' && (
        <div style={{
          fontSize: 8,
          color: C.gold,
          letterSpacing: 1,
          marginTop: 8,
          opacity: 0.7,
          fontStyle: 'italic',
        }}>
```

```
      ○ Artifact fracture will be seeded from session ID. Unreprodu
    </div>
  ))}
</div>

{/* Fields */}
{[
  { key: 'character', label: 'CHARACTER NAME', placeholder: 'ISOLDE
  { key: 'sessionId', label: 'SESSION ID', placeholder: 'SB001-ISO-
  { key: 'canonVersion', label: 'CANON VERSION', placeholder: '1.0'
].map(field => (
  <div key={field.key} style={{ marginBottom: 16 }}>
    <div style={{ fontSize: 7, color: C.chromeDim, letterSpacing: 2
      {field.label}
    </div>
    <input
      value={form[field.key]}
      onChange={e => setForm(f => ({ ...f, [field.key]: e.target.va
      placeholder={field.placeholder}
      style={{
        width: '100%',
        background: C.titanium,
        border: `1px solid ${C.chromeDim}`,
        borderRadius: 3,
        padding: '10px 12px',
        color: C.pearl,
        fontSize: 10,
        fontFamily: "'Courier New', monospace",
        letterSpacing: 1,
        outline: 'none',
        boxSizing: 'border-box',
      }}
    />
  </div>
))}

{/* Lane Status */}
<div style={{ marginBottom: 20 }}>
  <div style={{ fontSize: 7, color: C.chromeDim, letterSpacing: 2,
    LANE COMPLETION STATUS
  </div>
  <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', ga
    {Object.entries(laneLabels).map(([key, label]) => (
      <div
        key={key}
        onClick={() => setForm(f => ({
          ...f,
```

```
            laneStatus: { ...f.laneStatus, [key]: !f.laneStatus[key]
          }))}
          style={{
            display: 'flex',
            alignItems: 'center',
            gap: 8,
            background: form.laneStatus[key] ? 'rgba(68,255,170,0.06)
            border: `1px solid ${form.laneStatus[key] ? C.green : C.c
            borderRadius: 3,
            padding: '8px 12px',
            cursor: 'pointer',
            transition: 'all 0.2s',
          }}
        >
          <div style={{
            width: 8, height: 8,
            borderRadius: '50%',
            background: form.laneStatus[key] ? C.green : C.chromeDim,
            flexShrink: 0,
          }} />
          <div style={{
            fontSize: 7,
            color: form.laneStatus[key] ? C.green : C.chromeDim,
            letterSpacing: 1,
          }}>
            {label}
          </div>
        </div>
      ))}
    </div>
  </div>

  {/* Notes */}
  <div style={{ marginBottom: 24 }}>
    <div style={{ fontSize: 7, color: C.chromeDim, letterSpacing: 2,
      SESSION NOTES
    </div>
    <textarea
      value={form.notes}
      onChange={e => setForm(f => ({ ...f, notes: e.target.value }))}
      placeholder="Configuration notes, competing signals detected, g
      rows={4}
      style={{
        width: '100%',
        background: C.titanium,
        border: `1px solid ${C.chromeDim}`,
        borderRadius: 3,
```

```
            padding: '10px 12px',
            color: C.pearl,
            fontSize: 10,
            fontFamily: "'Courier New', monospace",
            letterSpacing: 1,
            outline: 'none',
            resize: 'vertical',
            boxSizing: 'border-box',
          }}
        />
      </div>

      <button
        onClick={handleSubmit}
        style={{
          background: C.gold,
          border: 'none',
          borderRadius: 3,
          padding: '12px 32px',
          color: C.void,
          fontSize: 9,
          letterSpacing: 4,
          cursor: 'pointer',
          fontFamily: "'Courier New', monospace",
          fontWeight: 'bold',
        }}
      >
        REGISTER · PASS QUALITY GATE
      </button>
    </div>
  )}

  {/* RECORD DETAIL */}
  {tab !== 'new' && selected && (
    <div>
      {/* Header */}
      <div style={{
        display: 'flex',
        alignItems: 'flex-start',
        justifyContent: 'space-between',
        marginBottom: 28,
        paddingBottom: 20,
        borderBottom: `1px solid ${C.chromeDim}`,
      }}>
        <div>
          <div style={{
            fontSize: 9,
```

```
        color: classColor,
        letterSpacing: 4,
        fontStyle: 'italic',
        marginBottom: 6,
      }}>
        {selected.class}
      </div>
      <div style={{
        fontSize: 24,
        color: C.white,
        letterSpacing: 4,
        marginBottom: 6,
      }}>
        {selected.character}
      </div>
      <div style={{
        fontSize: 8,
        color: C.chrome,
        letterSpacing: 2,
      }}>
        {selected.id} · SESSION {selected.sessionId} · v{selected.can
      </div>
    </div>

    {/* QR Code */}
    <div style={{ textAlign: 'center' }}>
      <QRDisplay
        data={JSON.stringify({ id: selected.id, fp: selected.fingerpr
        isArtifact={isArtifact}
        sessionId={selected.sessionId}
        size={90}
      />
      <div style={{
        fontSize: 6,
        color: classColor,
        letterSpacing: 2,
        marginTop: 6,
        fontStyle: 'italic',
      }}>
        {selected.class}
      </div>
    </div>
  </div>

  {/* Fingerprint */}
  <div style={{
    background: C.titanium,
```

```
  border: `1px solid ${C.chromeDim}`,
  borderRadius: 3,
  padding: '12px 16px',
  marginBottom: 20,
  display: 'flex',
  alignItems: 'center',
  justifyContent: 'space-between',
}}>
  <div style={{ fontSize: 7, color: C.chromeDim, letterSpacing: 2 }
    FINGERPRINT
  </div>
  <div style={{
    fontSize: 12,
    color: classColor,
    letterSpacing: 4,
    fontFamily: "'Courier New', monospace",
  }}>
    {selected.fingerprint}
  </div>
</div>

{/* Lane Status */}
<div style={{ marginBottom: 20 }}>
  <div style={{ fontSize: 7, color: C.chromeDim, letterSpacing: 2,
    LANE ARCHITECTURE
  </div>
  <div style={{ display: 'grid', gridTemplateColumns: 'repeat(4, 1f
    {Object.entries(laneLabels).map(([key, label]) => {
      const active = selected.laneStatus[key];
      return (
        <div key={key} style={{
          background: active ? 'rgba(68,255,170,0.06)' : C.titanium
          border: `1px solid ${active ? C.green : C.chromeDim}`,
          borderRadius: 3,
          padding: '10px 12px',
          textAlign: 'center',
        }}>
          <div style={{
            width: 8, height: 8,
            borderRadius: '50%',
            background: active ? C.green : C.chromeDim,
            margin: '0 auto 6px',
          }} />
          <div style={{
            fontSize: 6,
            color: active ? C.green : C.chromeDim,
            letterSpacing: 1,
```

```
            lineHeight: 1.4,
          }}>
            {label}
          </div>
        </div>
      );
    })}
  </div>
</div>

{/* Status Control */}
<div style={{ marginBottom: 20 }}>
  <div style={{ fontSize: 7, color: C.chromeDim, letterSpacing: 2,
    RELEASE STATUS
  </div>
  <div style={{ display: 'flex', gap: 8 }}>
    {['prerelease', 'released', 'archived'].map(s => (
      <button
        key={s}
        onClick={() => handleStatusChange(s)}
        style={{
          background: selected.status === s
            ? s === 'released' ? 'rgba(68,255,170,0.1)' : s === 'pr
            : C.titanium,
          border: `1px solid ${selected.status === s
            ? s === 'released' ? C.green : s === 'prerelease' ? C.g
            : C.chromeDim}`,
          color: selected.status === s
            ? s === 'released' ? C.green : s === 'prerelease' ? C.g
            : C.chromeDim,
          fontSize: 7,
          letterSpacing: 2,
          padding: '6px 14px',
          cursor: 'pointer',
          fontFamily: "'Courier New', monospace",
          borderRadius: 2,
          transition: 'all 0.2s',
        }}
      >
        {s.toUpperCase()}
      </button>
    ))}
  </div>
</div>

{/* Notes */}
<div style={{ marginBottom: 20 }}>
```

```
  <div style={{ fontSize: 7, color: C.chromeDim, letterSpacing: 2,
    SESSION NOTES
  </div>
  <div style={{
    background: C.titanium,
    border: `1px solid ${C.chromeDim}`,
    borderRadius: 3,
    padding: '12px 14px',
    fontSize: 9,
    color: C.chrome,
    letterSpacing: 0.5,
    lineHeight: 1.6,
  }}>
    {selected.notes || '—'}
  </div>
</div>

{/* Delta Log — Artifact only */}
{isArtifact && (
  <div style={{ marginBottom: 20 }}>
    <div style={{
      fontSize: 7,
      color: C.gold,
      letterSpacing: 2,
      marginBottom: 12,
      display: 'flex',
      alignItems: 'center',
      gap: 8,
    }}>
      CANON DELTA LOG
      <div style={{
        fontSize: 6,
        color: C.chromeDim,
        letterSpacing: 1,
      }}>
        · APPRECIATION RECORD · {selected.deltaLog.length} ENTRIES
      </div>
    </div>
    {selected.deltaLog.length === 0 ? (
      <div style={{
        fontSize: 8,
        color: C.chromeDim,
        letterSpacing: 1,
        fontStyle: 'italic',
      }}>
        No delta entries yet. Delta log populates automatically whe
      </div>
```

```
    ) : (
      selected.deltaLog.map((d, i) => <DeltaEntry key={i} delta={d}
    )}
  </div>
)}

{/* Card Mark Preview */}
<div style={{
  background: C.titanium,
  border: `1px solid ${classColor}`,
  borderRadius: 4,
  padding: 20,
  marginTop: 8,
}}>
  <div style={{ fontSize: 7, color: C.chromeDim, letterSpacing: 2,
    CARD MARK PREVIEW · PRINT QUALITY
  </div>
  <div style={{
    background: '#111218',
    borderRadius: 3,
    padding: 20,
    display: 'flex',
    alignItems: 'flex-end',
    justifyContent: 'space-between',
    position: 'relative',
    minHeight: 80,
    border: `1px solid ${C.chromeDim}`,
  }}>
    <div style={{
      fontSize: 7,
      color: C.chromeDim,
      letterSpacing: 2,
      fontStyle: 'italic',
    }}>
      [card image area]
    </div>

    {/* Bottom right mark — the signature */}
    <div style={{
      display: 'flex',
      flexDirection: 'column',
      alignItems: 'flex-end',
      gap: 4,
    }}>
      <QRDisplay
        data={JSON.stringify({ id: selected.id, fp: selected.finger
        isArtifact={isArtifact}
```

```
                    sessionId={selected.sessionId}
                    size={36}
                  />
                  <div style={{
                    fontSize: 7,
                    color: classColor,
                    letterSpacing: 2,
                    fontStyle: 'italic',
                  }}>
                    {selected.class}
                  </div>
                  <div style={{
                    fontSize: 5,
                    color: C.chromeDim,
                    letterSpacing: 1,
                  }}>
                    {selected.fingerprint}
                  </div>
                </div>
              </div>
            </div>

          </div>
        )}

      </div>
    </div>

    {/* Bottom classification band */}
    <div style={{ height: 4, background: `repeating-linear-gradient(90deg, ${C.
  </div>
);
}
```